SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1707**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I Kishore Kumar V(192521371) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Kishore V
Signature of the student:

Annexure A

Dry Run Evaluation – Detailed Answers

1. Question 1 – Breadth-First Search (BFS)

Given graph (tree): A is the root. A has two children, B and C. B has two children, D and E. C has two children, F and G.

Concept

Breadth-First Search explores a graph level by level. It uses a QUEUE (First-In-First-Out). The first node put in the queue is the first node taken out, so all nodes at depth d are visited before any node at depth d + 1. Here the levels are: Level 0 = A, Level 1 = B and C, Level 2 = D, E, F and G.

Algorithm

- Step 1: Put the start node A in the queue.
- Step 2: Remove the node at the front of the queue and mark it as visited.
- Step 3: Add all its unvisited children (left to right) to the back of the queue.
- Step 4: Repeat Steps 2 and 3 until the queue becomes empty.

Dry Run Table

Initial queue = [A]. The Queue column shows the queue after the node of that iteration has been visited and its children added.

Iteration	Queue	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	Empty	G

Iteration-wise Explanation

- Iteration 1: Remove A from the queue and visit it. A's children B and C are added. Queue = [B, C].
- Iteration 2: Remove B and visit it. B's children D and E are added at the back. Queue = [C, D, E].
- Iteration 3: Remove C and visit it. C's children F and G are added. Queue = [D, E, F, G].
- Iteration 4: Remove D and visit it. D has no children, so nothing is added. Queue = [E, F, G].

- Iteration 5: Remove E and visit it. E has no children.

Queue = [F, G].

- Iteration 6: Remove F and visit it. F has no children.

Queue = [G].

- Iteration 7: Remove G and visit it. G has no children.

Queue = [ ] (empty), so the search stops.

Answers to the Questions

1. BFS traversal order: A → B → C → D → E → F → G.

The nodes are visited level by level: level 0 (A), level 1 (B, C) and level 2 (D, E, F, G), each level from left to right.

2. Queue contents after each iteration:

Before iteration 1: [A]; after iteration 1: [B, C]; after 2: [C, D, E]; after 3: [D, E, F, G]; after 4: [E, F, G]; after 5: [F, G]; after 6: [G]; after 7: [ ].

Complexity of BFS

- Time complexity: O(b^d) – b is the branching factor (2 here) and d the depth of the goal.
- Space complexity: O(b^d) – all nodes of a level are kept in the queue.

- BFS is complete and optimal when every step has the same cost.

2. Question 2 – Depth-First Search (DFS)

The same graph is used: A → (B, C), B → (D, E), C → (F, G).

Concept

Depth-First Search goes as deep as possible along one branch before backtracking. It uses a STACK (Last-In-First-Out) or recursion. To visit the left child first, the children are pushed in reverse order (right child first, then left child), so the left child ends on top of the stack.

Algorithm

- Step 1: Push the start node A on the stack.
- Step 2: Pop the top node and mark it as visited.
- Step 3: Push its unvisited children on the stack (right child first, so the left child is on top).
- Step 4: Repeat Steps 2 and 3 until the stack becomes empty.

Dry Run Table

The top of the stack is written on the left.

Iteration	Stack	Visited Node
1	B, C	A
2	D, E, C	B
3	E, C	D
4	C	E
5	F, G	C
6	G	F
7	Empty	G

Iteration-wise Explanation

- Iteration 1: Pop A and visit it. Push C and then B, so the stack is [B, C] with B on top.
- Iteration 2: Pop B and visit it. Push E and then D. Stack = [D, E, C].
- Iteration 3: Pop D and visit it. D is a leaf, so nothing is pushed. Stack = [E, C].
- Iteration 4: Pop E and visit it. E is a leaf. Stack = [C]. The left subtree of A is finished, so the search backtracks to C.
- Iteration 5: Pop C and visit it. Push G and then F. Stack = [F, G].

- Iteration 6: Pop F and visit it. F is a leaf. Stack = [G].
- Iteration 7: Pop G and visit it. The stack is empty, so the search stops.

DFS traversal order: A → B → D → E → C → F → G.

Comparison of BFS and DFS

Point	**BFS**	**DFS**
Data structure	Queue (FIFO)	Stack (LIFO)
Order here	A B C D E F G	A B D E C F G
Memory	O(b^d), large	O(b·m), small
Shortest path	Yes (equal step costs)	Not guaranteed
Completeness	Yes	No for infinite depth

3. Question 3 – 8-Puzzle using BFS

Initial state: 1 3 6 / 5 0 2 / 4 7 8. Goal state: 1 2 3 / 4 5 6 / 7 8 0. (0 represents the blank tile.)

Problem Formulation

- States: every arrangement of the tiles 1–8 and the blank on the 3 × 3 board.
- Initial state: 136/502/478 (rows separated by a slash).

- Actions: move the blank Up, Down, Left or Right (when it stays inside the board).
- Goal test: the board equals 123/456/780.
- Path cost: every move costs 1, so the cost of a path equals the number of moves.

Method Used

BFS keeps a queue of states. In each iteration the first state in the queue is removed and expanded: its successor states are generated by moving the blank in the order Up, Down, Left, Right. A state that has already been generated is not added again, which avoids loops and repeated work. The search stops when the goal state is taken from the queue.

Dry Run Table (first five iterations)

Iteration	**Current State**	**Queue Before Expansion**	**Queue After Expansion**
1	136/502/478	136/502/478	106/532/478, 136/572/408, 136/052/478, 136/520/478

2	106/532/478	106/532/478, 136/572/408, 136/052/478, 136/520/478	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478
3	136/572/408	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480
4	136/052/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078

5	136/520/478	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078	016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470

Iteration-wise Expansion

- Iteration 1: Expand 136/502/478 (depth 0). The blank can move Up, Down, Left, Right. New states added to the queue: 106/532/478 (blank Up), 136/572/408 (blank Down), 136/052/478 (blank Left), 136/520/478 (blank Right).
- Iteration 2: Expand 106/532/478 (depth 1). The blank can move Down, Left, Right. New states added to the queue: 016/532/478 (blank Left), 160/532/478 (blank Right).
- Iteration 3: Expand 136/572/408 (depth 1). The blank can move Up, Left, Right. New states added to the queue: 136/572/048 (blank Left), 136/572/480 (blank Right).

- Iteration 4: Expand 136/052/478 (depth 1). The blank can move Up, Down, Right. New states added to the queue: 036/152/478 (blank Up), 136/452/078 (blank Down).
- Iteration 5: Expand 136/520/478 (depth 1). The blank can move Up, Down, Left. New states added to the queue: 130/526/478 (blank Up), 136/528/470 (blank Down).

The search continues in the same way, level by level, until the goal state is removed from the queue.

Number of States at Each Depth

Depth	**States generated**	**Note**
0	1	Initial state
1	4	
2	8	
3	8	
4	16	
5	32	
6	60	
7	72	
8	136	Goal found

9	163	Partly generated

The goal first appears at depth 8, and all states of depth 8 are generated before the goal is taken from the queue. The depth 9 states in the table are only those generated while the depth 8 states were being expanded, before the goal was reached.

Answers to the Questions

1. Which node is expanded in each iteration?

Iteration 1 expands the initial state 136/502/478. Iteration 2 expands 106/532/478, iteration 3 expands 136/572/408, iteration 4 expands 136/052/478 and iteration 5 expands 136/520/478. These are the four children of the initial state, taken in the order Up, Down, Left, Right. After that the depth 2 states are expanded, then depth 3, and so on until depth 8. With this move order the goal state is the 311-th state taken from the queue.

2. How many states are generated in total?

500 states are generated in total (including the initial state) before the goal state is removed from the queue. The count depends on the order in which the blank is moved and on repeated states being ignored.

3. Complete solution path from the initial state to the goal state

The blank is moved 8 times: Right, Up, Left, Down, Left, Down, Right, Right.

Initial state: 1 3 6 5 0 2 4 7 8 (rows: 136/502/478)

Step 1 (blank Right): 1 3 6 5 2 0 4 7 8 (rows: 136/520/478)

Step 2 (blank Up): 1 3 0 5 2 6 4 7 8 (rows: 130/526/478)

Step 3 (blank Left): 1 0 3 5 2 6 4 7 8 (rows: 103/526/478)

Step 4 (blank Down): 1 2 3 5 0 6 4 7 8 (rows: 123/506/478)

Step 5 (blank Left): 1 2 3 0 5 6 4 7 8 (rows: 123/056/478)

Step 6 (blank Down): 1 2 3 4 5 6 0 7 8 (rows: 123/456/078)

Step 7 (blank Right): 1 2 3 4 5 6 7 0 8 (rows: 123/456/708)

Step 8 (blank Right) – Goal: 1 2 3 4 5 6 7 8 0 (rows: 123/456/780)

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

BFS expands states in increasing order of depth: every state at depth d is expanded before any state at depth d + 1. In the 8-puzzle every move has the same cost (1), so the cost of a path is exactly its depth. Therefore the first time the goal is reached, it has been reached at the smallest possible depth; no shorter path can exist, otherwise the goal would have been found at an earlier level. BFS is also complete because the number of states is finite (9!/2 = 181,440 reachable states) and repeated states are avoided. The price is memory: the queue holds a whole level of states, so the space needed is O(b^d).

Conclusion

BFS solves this 8-puzzle in the minimum of 8 moves. It is optimal and complete for unit-cost problems, but its time and memory grow exponentially with depth, so informed methods such as A* are preferred for harder puzzles.